

Security & Client-Side Privacy Specification

Source: docs/SECURITY_PRIVACY.md • Status: Verified & Synchronized

Product: Cubyntra
Organization: Necookie Labs
Status: Implemented
Scope: Client-Side Data Privacy, Camera Permissions, and Web Security

1. Core Security & Privacy Philosophy

Cubyntra is designed with a strict Zero-Knowledge, Client-Side Privacy Model. Users pointing their mobile or desktop camera at a physical object must have unconditional assurance that their video feed, environment, and physical likeness never leave their device.

Core Privacy Commitments:

- Zero Video Uploads: Video streams, canvas image buffers, and raw pixel data never leave local memory.
- Zero Cloud Vision Processing: Computer vision, sticker extraction, and color space classification occur 100% on the local CPU/GPU.
- Hardware Stream Teardown: Camera tracks are explicitly terminated (`track.stop()`) immediately upon unmounting or when the user navigates away, guaranteeing the webcam hardware indicator turns off.
- No Third-Party Analytics Trackers: No third-party tracking scripts, cookies, or telemetry libraries are bundled.

2. Browser Camera Permissions & Hardware Lifecycle

2.1 WebRTC Secure Context Requirements

Browsers enforce that `navigator.mediaDevices.getUserMedia` is only accessible within a Secure Context (`https://` or `http://localhost`).

- Cubyntra enforces HTTPS in production environments.
- In insecure environments (`http://` on LAN/WAN), camera activation is gracefully disabled with user guidance to enable HTTPS or use manual scramble mode.

2.2 Stream Acquisition and Teardown

```
// Safe acquisition with ideal mobile resolution
const stream = await navigator.mediaDevices.getUserMedia({
  video: {
    facingMode: 'environment', // Rear camera preferred on mobile
    width: { ideal: 1280 },
    height: { ideal: 720 },
  },
  audio: false, // Audio is NEVER requested
});

// Explicit cleanup hook
export function stopCameraStream(stream: MediaStream | null) {
  if (!stream) return;
  stream.getTracks().forEach((track) => {
    track.stop(); // Terminates sensor hardware immediately
  });
}
```

3. Threat Modeling & Vulnerability Mitigation

Threat Vector	Severity	Mitigation Strategy
Exfiltration of Video Data	Critical	Architecture contains zero network egress endpoints for media. All canvas buffers are held in scoped memory and garbage collected.
Denial of Service via Solver ReDoS / Hang	Medium	Solver engine implements a strict 5,000ms execution timeout cap. If search exceeds threshold, it cleanly aborts.
Arbitrary Code Injection via Scramble String	High	Input strings are strictly validated against <code>^[UDFBLR]{54}</code> character whitelists before parsing. No <code>eval()</code> or dynamic code evaluation exists.
Memory Exhaustion (Three.js WebGL Leaks)	Medium	Comprehensive <code>dispose()</code> hooks for all geometries, materials, and textures attached to component unmount lifecycle.
XSS via UI Inputs	High	React 19 automatic JSX string escaping; strict TypeScript types; no <code>dangerouslySetInnerHTML</code> usage.

4. Content Security Policy (CSP) Guidelines

For production deployments, the following HTTP response headers are recommended:

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob:; connect-src 'self'; media-src 'self' blob:; object-src 'none'; frame-ancestors 'none';
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(self), microphone=(), geolocation=()
```

(Notice that microphone and geolocation are explicitly forbidden by Permissions-Policy, while camera is restricted exclusively to self).